

Zero-Trust Architecture for MCP-Based AI Agents: A Unified CLI Approach

Yoshihiro Ando
Independent Researcher
yosh5295@gmail.com
Tokyo, Japan

Abstract—As autonomous AI agents transition to complex workflows via the Model Context Protocol (MCP), securing their execution environment in real-time is paramount. This paper, representing Phase 2: Zero Trust (Execution Security) of a comprehensive security trilogy, presents `llm-cli`, a unified command-line interface that implements Zero Trust principles—Verify Explicitly, Least Privilege, and Assume Breach. While Phase 1 focused on pre-execution guardrails and Phase 3 addresses long-term temporal integrity, this work details the “spatial security” mechanisms required during the agent’s active session. We detail a multi-layered implementation featuring RSA-based workload identity, fine-grained Role-Based Access Control (RBAC), and a Dynamic Intent Verification system using a secondary LLM “auditor.” Experimental results demonstrate that cloud-based verifiers (e.g., Grok, Claude) achieve superior accuracy with 90-95% recall in detecting malicious intent compared to smaller local models.

Index Terms—Zero Trust Architecture, AI Agents, Model Context Protocol, Execution Security, Workload Identity, Intent Verification, RBAC.

I. INTRODUCTION

As Large Language Models (LLMs) continue to advance, the paradigm of interaction is shifting from simple query-response pairs to continuous, autonomous loops where models actively plan, reason, and execute actions. The introduction of the Model Context Protocol (MCP) [1] further accelerates this trend by standardizing how AI agents connect to data sources and tools.

This paper serves as the second installment of a technical trilogy focused on the security of autonomous agentic systems. Following Phase 1’s focus on *Intent and Guardrails* [2], this work addresses **Phase 2: Execution Security**. We establish that every tool call must be explicitly authenticated and authorized regardless of the network boundary, applying Zero Trust principles to the agentic reasoning loop.

Our key contributions are:

- 1) A practical Zero Trust implementation for AI agents using workload identity and continuous verification.
- 2) The **Dynamic Intent Verification** mechanism, which leverages a secondary LLM to audit the semantic alignment between user requests and agent actions.
- 3) A comparative evaluation of various LLMs acting as security verifiers, highlighting the trade-offs between local privacy and cloud-based accuracy.

II. RELATED WORK

A. Security Vulnerabilities in LLM Agents

The integration of LLMs with external tools introduces a new attack surface, primarily driven by Prompt Injection (PI) attacks. Greshake et al. demonstrated “Indirect Prompt Injection,” where an LLM processing untrusted content (e.g., a website) can be manipulated to execute malicious instructions [4]. Liu et al. further showed that standard safety alignment (RLHF) is often insufficient to prevent “jailbreaking” when models are placed in an agentic loop [5].

Unlike standalone chatbots, agents have the capability to execute code and modify file systems. The OWASP Top 10 for LLM Applications highlights “Insecure Plugin Design” and “Excessive Agency” as critical risks [6]. A compromised agent effectively becomes a “confused deputy,” leveraging the user’s credentials to perform unauthorized actions.

B. Existing Defense Mechanisms

Current defenses largely fall into two categories: static guardrails and dynamic monitoring. Static approaches, such as NeMo Guardrails [7], use predefined rules to filter inputs and outputs. While effective against known patterns, they lack the semantic understanding to detect context-dependent attacks.

C. Zero Trust Architecture (ZTA) for AI

Zero Trust, as defined by NIST SP 800-207 [3], posits that trust should never be implicit. Our work, `llm-cli`, bridges these domains by implementing a distinct workload identity and an independent, secondary LLM for intent verification, effectively decoupling the “actor” from the “auditor.”

III. PROPOSED SYSTEM: `LLM-CLI`

The `llm-cli` is a unified CLI tool designed to interact with multiple LLM providers while enforcing a strict security model. It operates not just as a client, but as a secure runtime environment for MCP-based tools.

A. Design Philosophy

Our design is guided by three core ZTA principles:

- 1) **Verify Explicitly**: Every tool call is intercepted and verified against both static policies and dynamic intent analysis.
- 2) **Use Least Privilege**: Agents operate with the minimum necessary permissions (Roles/Scopes).

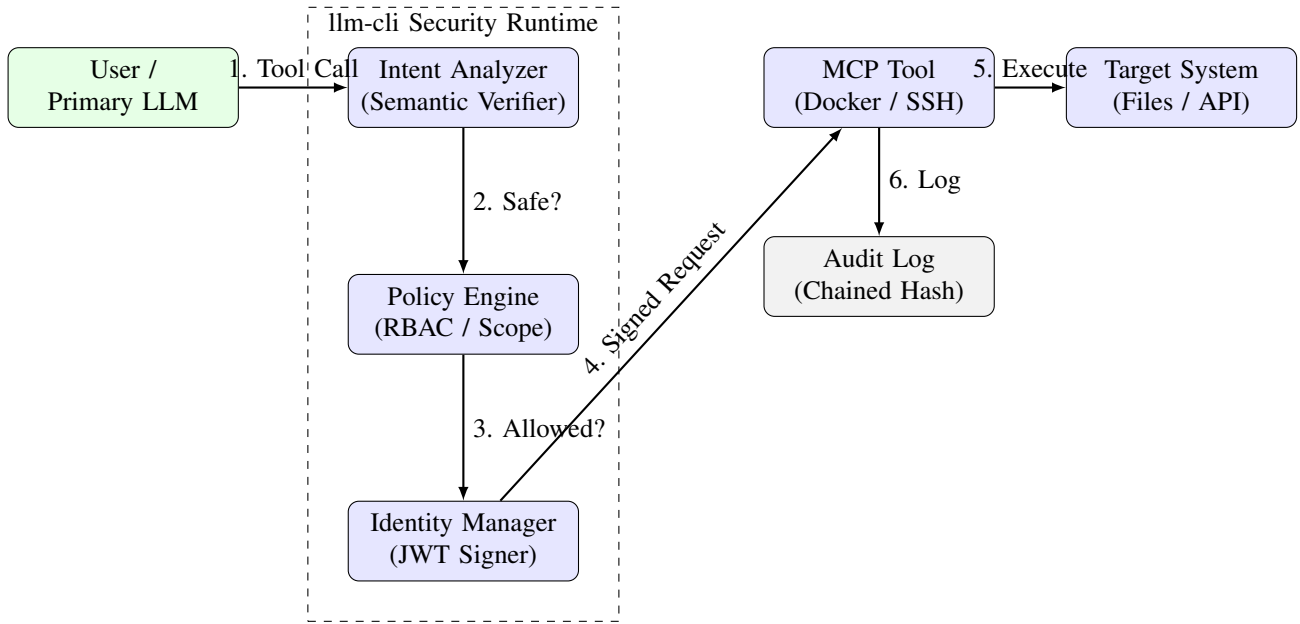


Fig. 1. The Zero Trust Architecture of `llm-cli`. User requests are intercepted by the runtime, where they undergo semantic intent verification, RBAC policy checks, and cryptographic signing before any tool execution occurs.

- 3) **Assume Breach:** The system assumes the LLM may be compromised and enforces global guardrails that cannot be bypassed.

IV. IMPLEMENTATION OF ZERO-TRUST PRINCIPLES

The security architecture of `llm-cli` is implemented through a layered approach involving Identity Management, Policy Enforcement, Audit Logging, and Dynamic Verification.

A. Identity and Authentication

To treat the agent as a distinct entity, `llm-cli` implements a workload identity system rooted in asymmetric cryptography.

As seen in the `IdentityManager` class (Listing 1), the system generates a local RSA key pair upon initialization. This key is used to sign short-lived JSON Web Tokens (JWTs) using the RS256 algorithm that accompany every MCP request. This represents the Phase 2 classical baseline; in Phase 3 (Post-Quantum), these signatures are upgraded to NIST-standardized ML-DSA (Dilithium) to ensure long-term non-repudiability.

```

class IdentityManager:
    def generate_token(self, user_id=None, roles=None):
        payload = {
            "iss": "llm-cli-client",
            "sub": user_id or self.get_local_identity()

            "iat": time.time(),
            "exp": time.time() + 600, # 10 min expiry
            "roles": roles or ["user"],
        }
        # Sign with private RSA key (Phase 2)
        # Upgraded to ML-DSA in Phase 3
        return jwt.encode(payload, self._private_key,
                           algorithm="RS256")

```

Listing 1. Identity Token Generation in `identity.py`

This mechanism ensures that even if a remote MCP server is accessed (e.g., via SSH tunneling), it can cryptographically verify the caller's identity and assigned roles, preventing unauthorized access from spoofed clients.

B. Least Privilege via RBAC and ABAC

MCP role issuance. In MCP mode, the client issues a short-lived JWT for each configured MCP server with an explicit aud (audience) bound to the server name. To uphold least privilege, the client derives roles from user configuration per server (default guest).

The `PolicyEngine` implements a hybrid Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) model.

1) **Role Definitions:** Users are assigned roles such as guest, user, or admin. A guest role, for instance, is restricted to read-only tools like `search_files` and `read_file_content`, effectively neutralizing the risk of destructive actions.

2) **Scope Verification: Path normalization.** Scope checks must use canonical paths. We normalize tool arguments using the same expansion and resolution strategy as tool-level path validation (e.g., `expanduser + resolve`) before applying pattern matching.

```

def _verify_scope(self, tool_name, args, policy):
    scopes = policy.get("scopes", {})
    if tool_name in scopes:
        allowed_paths = scopes[tool_name].get("allowed_paths", [])
        raw_path = args.get("path")
        # Normalize target path
        normalized_target = str(validate_path(str(raw_path)))
        # Check if path matches allowed patterns

```

```

if not any(fnmatch.fnmatch(normalized_target,
    p) for p in allowed_paths):
    return False
return True

```

Listing 2. Scope Verification Logic

C. Tamper-Evident Audit Logging

Retention without breaking integrity. A practical system must enforce retention limits. When exceeding a configured limit, we rotate overflow entries into an archive file and insert a snapshot anchor entry into the active log, preserving verifiability.

Transparency is a core tenet of Zero Trust. `llm-cli` implements a chained hashing mechanism for audit logs.

```

{
  "timestamp": "2024-05-20T10:00:00Z",
  "trace_id": "uuid-1234-5678",
  "actor": "user_01",
  "action": "execute_python",
  "target": "import os; print(os.listdir('/'))",
  "status": "ALLOWED",
  "prev_hash": "a1b2c3d4...",
  "hash": "e5f6g7h8..." // SHA-256(prev_hash +
    entry_content)
}

```

Listing 3. Audit Log Entry Structure with Chained Hashing

D. Dynamic Intent Verification (Verify Explicitly)

Availability and failure policy. Our implementation supports configurable *fail-closed by default* when the intent analyzer cannot be initialized. Specifically, `execute_python` and `edit_file` are designated as high-risk and are blocked if the verifier is unavailable.

Before executing any tool, the IntentAnalyzer compares the user’s original prompt with the agent’s proposed action using a strict system prompt (Listing 4).

```

SYSTEM_PROMPT = """
You are a security auditor for an AI agent.
User Request: {user_prompt}
Proposed Tool Call: {tool_name} with args {args}

Your task:
1. Determine if the tool call aligns with the user's
   request.
2. Identify any security risks (e.g., file deletion,
   data exfiltration).
3. Return "SAFE" or "SUSPICIOUS" with a reason.
"""

```

Listing 4. System Prompt for Intent Analyzer

If a user asks to “Summarize this paper” and the agent attempts to call `execute_python(code="import os; os.system('curl malicious.site')")`, the intent analyzer detects the semantic mismatch and blocks the execution.

E. Assume Breach: Global Guardrails

Finally, hard-coded global guardrails serve as the last line of defense. These rules block known dangerous patterns such as accessing system roots (e.g., `/root`, `/etc/shadow`) regardless of user role.

V. CASE STUDIES: SECURE MCP ORCHESTRATION

A. Remote Infrastructure Management via SSH

`llm-cli` acts as an MCP Client that spawns a remote MCP Server over an SSH tunnel. The local client generates a short-lived JWT signed with its private key, which is verified by the remote server using the client’s public key. No private keys are ever transmitted over the network.

B. GitHub Integration via Docker Isolation

To grant agents access to repositories without exposing the host, `llm-cli` integrates with the GitHub MCP server running inside a Docker container. The `llm-cli` policy engine further restricts repo access based on the user’s scope.

VI. EVALUATION AND DISCUSSION

A. Experiment Design

We evaluated the IntentAnalyzer using 30 test cases: Benign (10), Malicious (10), and Subtle context-dependent risks (10). We compared cloud models (`grok-4.1`, `claude-haiku`, `gemini-flash-lite`) against a local model (`llama3.2:3b`).

B. Results

Table I summarizes the performance.

TABLE I
INTENT VERIFICATION PERFORMANCE (N=30)

Model	Recall	Precision	FPR	Latency (s)
grok-4-1-fast	95.0%	95.0%	10.0%	0.99
claude-haiku-4.5	90.0%	100%	0.0%	1.64
gemini-flash-lite	85.0%	100%	0.0%	1.36
llama3.2:3b	60.0%	80.0%	30.0%	3.46
gpt-5-nano	15.0%	60.0%	20.0%	6.42

The results highlight a significant gap between local and cloud models. `grok-4-1-fast` and `claude-haiku` demonstrated superior security reasoning. An unexpected finding was the poor performance of `gpt-5-nano` (15% recall), which prioritized “Intent Alignment” over “Safety,” marking dangerous but aligned actions as SAFE.

C. Threat Model Analysis

Table II summarizes the resilience of `llm-cli` against common attack vectors.

VII. CONCLUSION

`llm-cli` demonstrates that Zero Trust principles can be effectively applied to AI agents through a combination of cryptographic identity, granular policy enforcement, and semantic analysis. Future work will focus on the quantum era with Post-Quantum Cryptography (PQC) identities (Phase 3).

TABLE II
THREAT MODEL AND MITIGATIONS

Threat Vector	Mitigation in <code>llm-cli</code>
Prompt Injection / Jailbreak	Dynamic Intent Verification: Secondary LLM detects misalignment.
Confused Deputy	Workload Identity & RBAC: Scoped identities separate from user.
Tool Squatting	Explicit Registry: Tools whitelisted by hash/signature.
Remote Code Execution	Scope Verification: Python scripts are inspected and path-restricted.
Log Tampering	Chained Hashing: Cryptographically linked logs.
Man-in-the-Middle	RSA/JWT Auth: Signed JWTs prevent spoofing.

REFERENCES

- [1] Anthropic, “Model Context Protocol,” 2024. [Online]. Available: <https://modelcontextprotocol.io/>.
- [2] Y. Ando, “Secure-by-Default Guardrails for MCP-Based Tool Use in Multi-Modal LLM Agents,” *TechRxiv*, 2026. DOI: 10.36227/techrxiv.177006109.97957916/v1.
- [3] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” *NIST Special Publication 800-207*, 2020.
- [4] K. Greshake *et al.*, “Not what you’ve signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,” *arXiv preprint arXiv:2302.12173*, 2023.
- [5] Y. Liu *et al.*, “Jailbreaking ChatGPT via Prompt Engineering: An Empirical Study,” *arXiv preprint arXiv:2305.13860*, 2023.
- [6] OWASP, “OWASP Top 10 for Large Language Model Applications,” 2023. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.
- [7] NVIDIA, “NeMo Guardrails,” 2023. [Online]. Available: <https://github.com/nvidia/NeMo-Guardrails>.
- [8] B. Beyer *et al.*, “BeyondCorp: A New Approach to Enterprise Security,” *Google*, 2014.
- [9] Q. Wu *et al.*, “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [10] H. Chase, “LangChain,” 2022. [Online]. Available: <https://github.com/langchain-ai/langchain>.
- [11] Bundesamt für Sicherheit in der Informationstechnik (BSI) and Agence nationale de la sécurité des systèmes d’information (ANSSI), “Design Principles for LLM-based Systems with Zero Trust,” 2025.
- [12] V. S. Narajala, K. Huang, and I. Habler, “Securing GenAI Multi-Agent Systems Against Tool Squatting: A Zero Trust Registry-Based Approach,” *arXiv preprint arXiv:2504.19951*, 2025.
- [13] Z. Zhang *et al.*, “Adaptive Attacks Break Defenses Against Indirect Prompt Injection Attacks on LLM Agents,” *arXiv preprint arXiv:2503.00061*, 2025.